

Design for class “SampleClass1”

<SampleClass1 class image in UML>

Table 1. Attribute design

#	Name	Data type	Default value	Description
1				
2				
...				

Table 2. Operation design

#	Name	Return type	Description (purpose)
1			
2			
...			

Parameter:

- x: Default value, description
- y: Default value, description
- ...

Exception:

- AException if ...
- BException if ...
- ...

Method:

- How to use parameters / attributes

Other diagrams:

- Flowchart / Activity diagram / Sequence diagram if the method has a complex / special algorithm
- State diagram if any

GIỮ NGUYÊN TRANG NÀY ĐỂ COPY LÀM TEMPLATE

Design for class “Order”

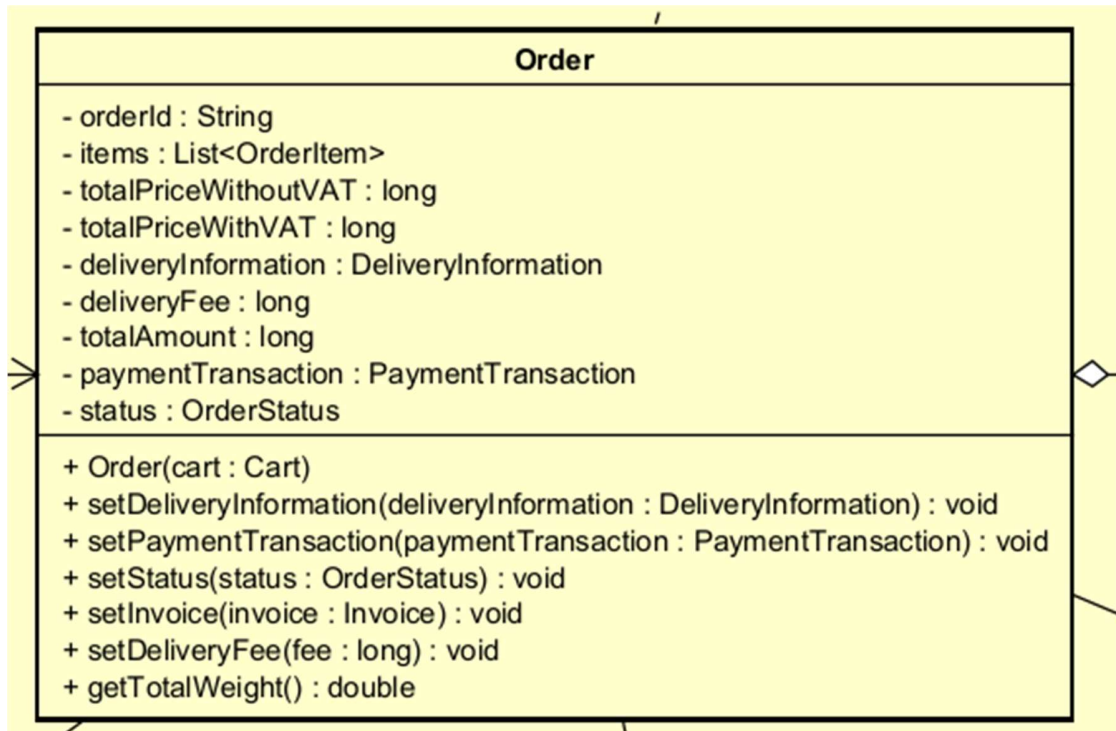


Table 1. Attribute design

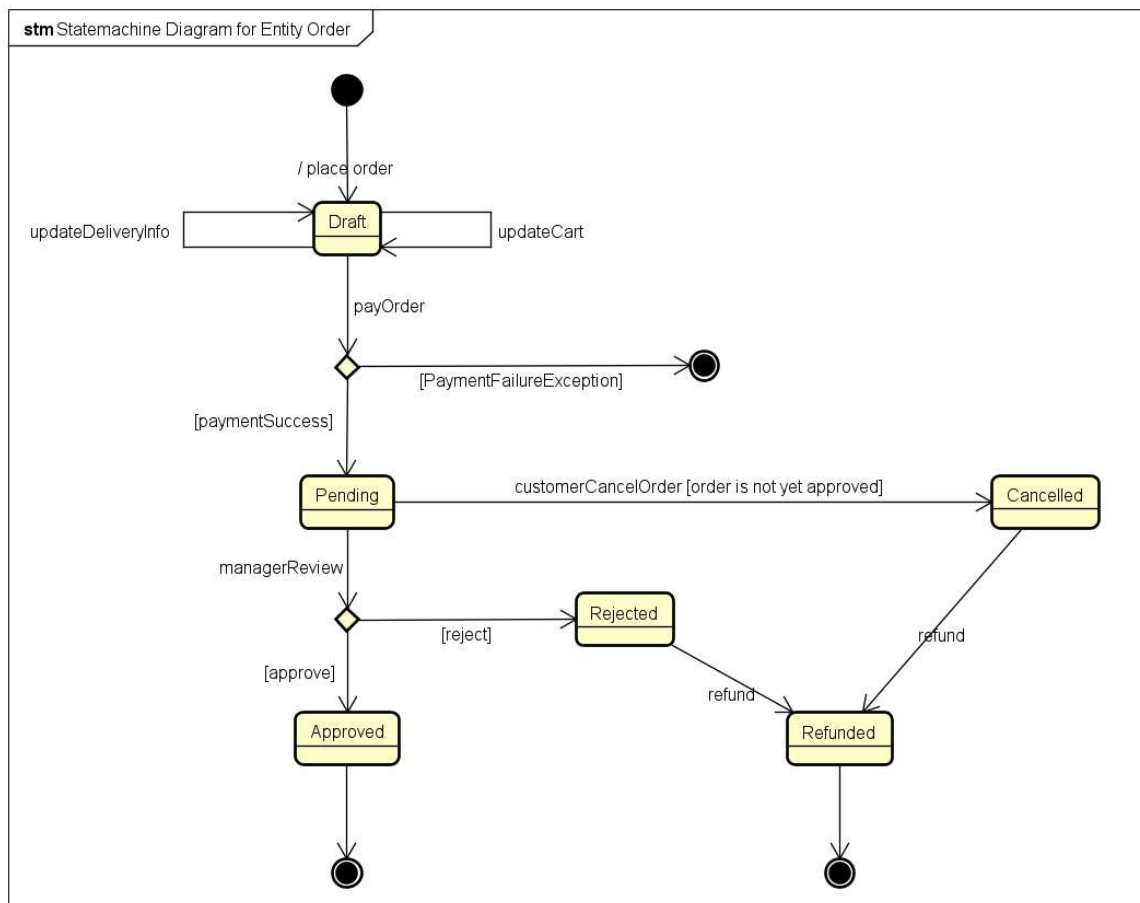
#	Name	Data type	Default value	Description
1	orderId	String		Unique ID for an order, generated automatically
2	items	List<OrderItem>		List of items of the order
3	totalPriceWithoutVAT	long		Total price of the order without VAT
4	totalPriceWithVAT	long		Total price of the order with VAT
5	deliveryInformation	DeliveryInformation	null	The delivery information of the order
6	deliveryFee	long		Calculated delivery fee
7	totalAmount	long		totalPriceWithVAT + deliveryFee

8	paymentTransaction	PaymentTransaction	null	The transaction information of the order
9	status	OrderStatus	Draft	The status of the order

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	Order		Construct the order entity based on the cart content
2	setDeliveryInformation	void	Set the delivery information entity
3	setPaymentTransaction	void	Set the payment transaction entity
4	setStatus	void	Set order status
5	setDeliveryFee	void	Set the delivery fee of the order
6	getTotalWeight	double	Get the total weight of all items in the order

State machine diagram:



Design for class “OrderItem”

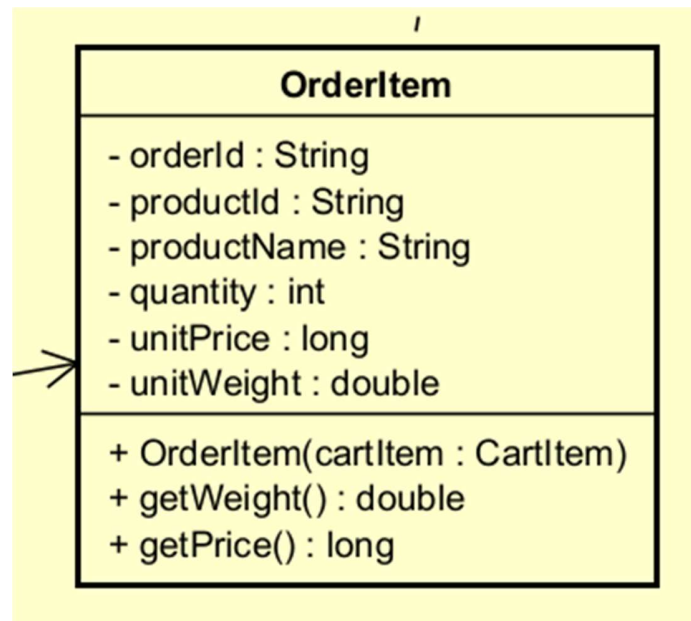


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderId	String		The ID of the order the item belongs to
2	productId	String		The ID of the product it references
3	productName	String		The title of the product it references
4	quantity	int		The quantity of the item
5	unitPrice	long		The price of 1 product item
6	unitWeight	double		The weight of 1 product item

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	OrderItem		Construct an order item based on the cart item
2	getWeight	double	Get the total weight (unitWeight * quantity) of the item
3	getPrice	double	Get the total price (unitPrice * quantity) of the item

Design for class “DeliveryInformation”

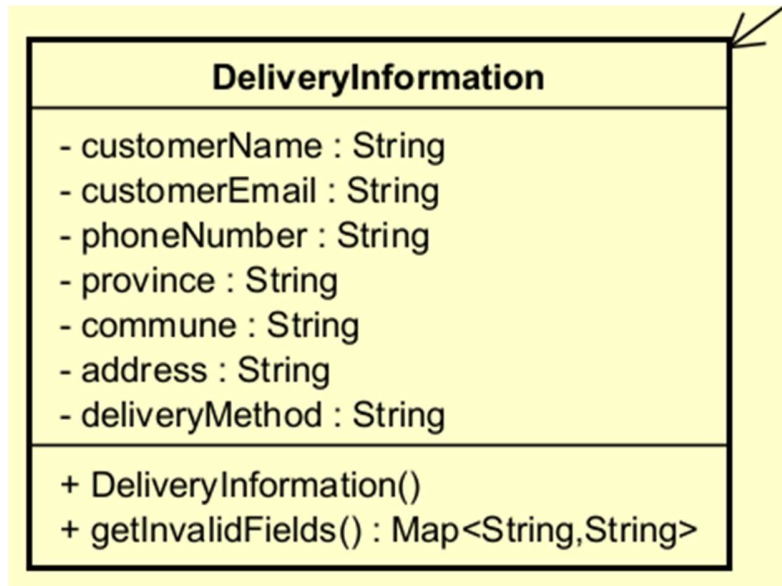


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	customerName	String	null	Name of customer
2	customerEmail	String	null	Email of customer
3	phoneNumber	String	null	Phone number of customer
4	province	String	null	The province the customer lives in
5	commune	String	null	The commune the customer lives in
6	address	String	null	More detailed address
7	deliveryMethod	String	null	The instruction/method on how to ship the order

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	DeliveryInformation		Create a delivery information entity
2	getInvalidFields	Map<String, String>	Return a map from the field name to the message of why that field is invalid; the return type is to suitable to display what went wrong on UI.

Design for class “PaymentTransaction”

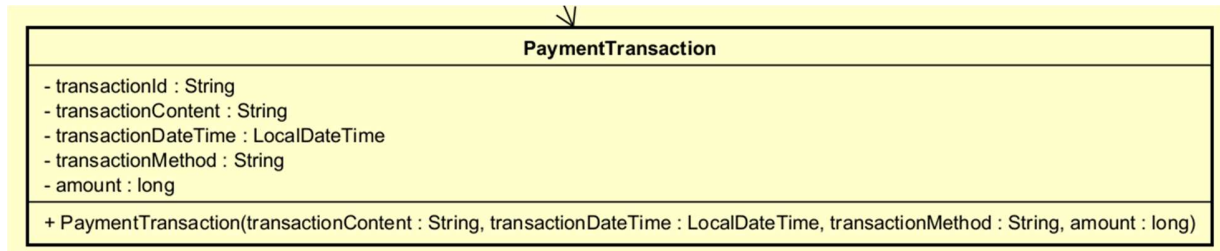


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	transactionId	String		The unique ID of the transaction, generated automatically
2	transactionContent	String		The content / message of the transaction
3	transactionDateTime	LocalDateTime		The timestamp of the transaction
4	transactionMethod	String		Which payment method is used to pay order
5	amount	long		The amount paid

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	PaymentTransaction		Construct a payment transaction entity

Design for class “Invoice”

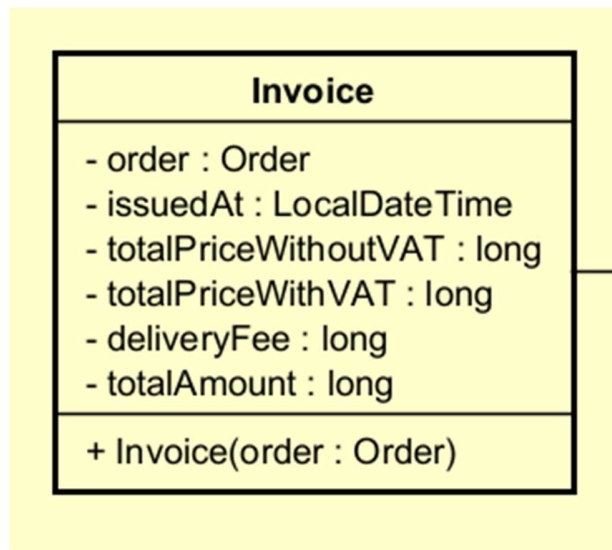


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	order	Order		The order that the invoice references
2	issuedAt	LocalDateTime		The timestamp at which the invoice is created
3	totalPriceWithoutVAT	long		Total price of the order without VAT
4	totalPriceWithVAT	long		Total price of the order with VAT
5	deliveryFee	long		Calculated delivery fee
6	totalAmount	long		totalPriceWithVAT + deliveryFee

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	Invoice		Construct an invoice class

Design for class “Cart”

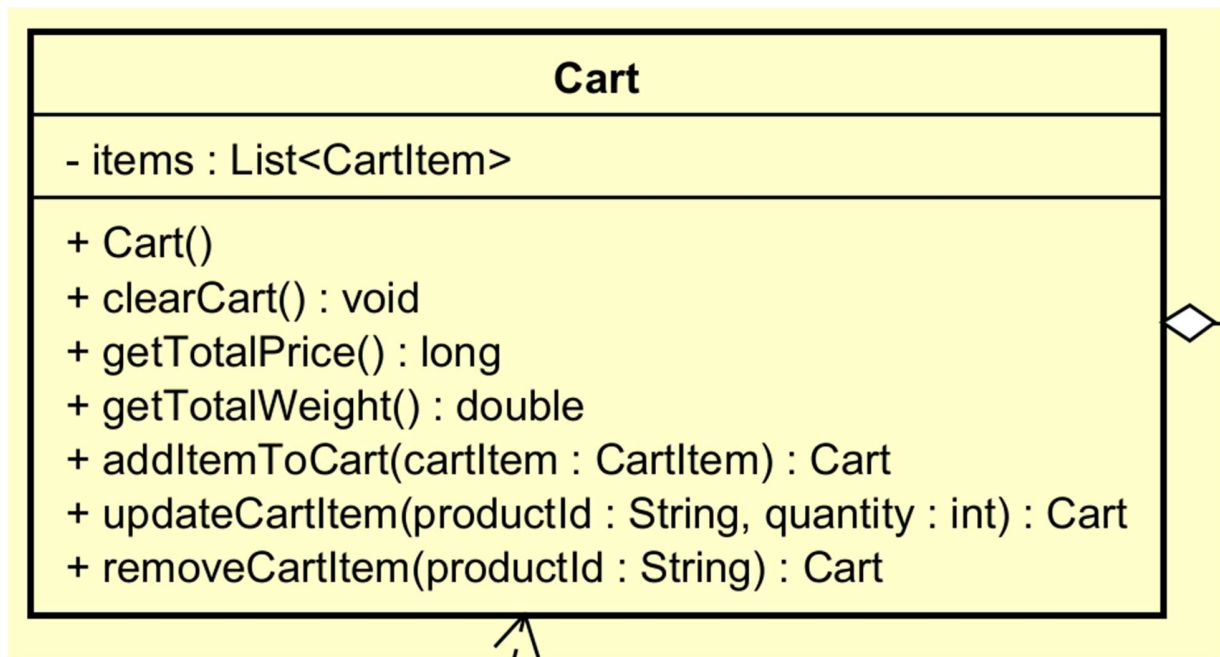


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	items	List<CartItem>	List<>()	The list of items in cart

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	Cart		Construct a cart
2	clearCart	void	Clear cart content
3	getTotalPrice	long	Get the total price of the cart
4	getTotalWeight	double	Get the total weight of the cart
5	addItemToCart	Cart	Add a new cart item, return itself
6	updateCartItem	Cart	Update an item's quantity in cart; if the quantity is 0 the item is removed; return itself
7	removeCartItem	Cart	Remove an item from cart, return itself

Parameter:

- addItemToCart(cartItem : CartItem) : Cart
 - o cartItem: the item to be added
- updateCartItem(productId : String, quantity : int) : Cart

- productId: the ID of the product to be updated (searched in cart)
 - quantity: the quantity to be updated
- removeCartItem(productId : String)
 - productId: the ID of the product to be removed

Design for class “CartItem”

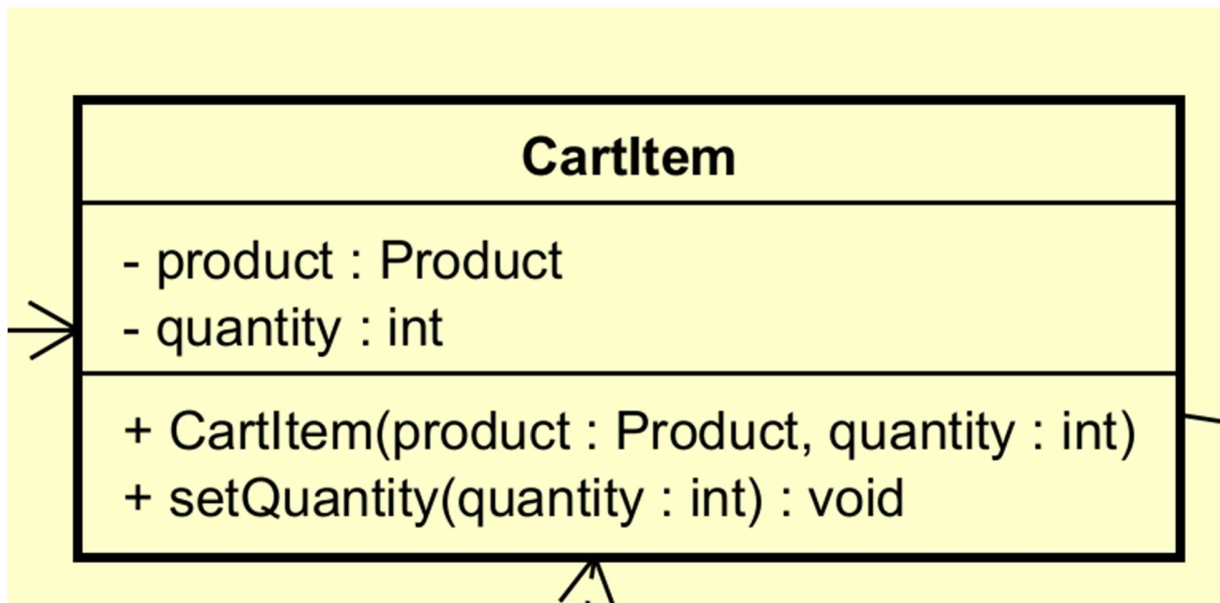


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	product	Product		The product the cart item references
2	quantity	int		The quantity of product

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	CartItem		Construct a cart item
2	setQuantity	void	Set the quantity of the product

Design for class “User”

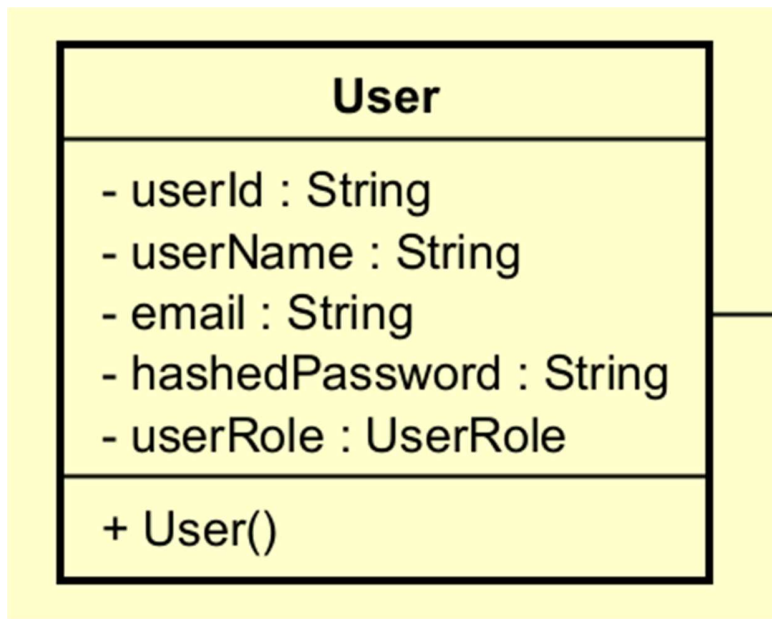


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	userId	String		The unique ID of the user, generated automatically
2	userName	String		Name of the user
3	email	String		Email of the user
4	hashedPassword	String		Password of the user account, securely hashed
5	userRole	UserRole		The role of the user

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	User		Construct a user

Design for class “Product”

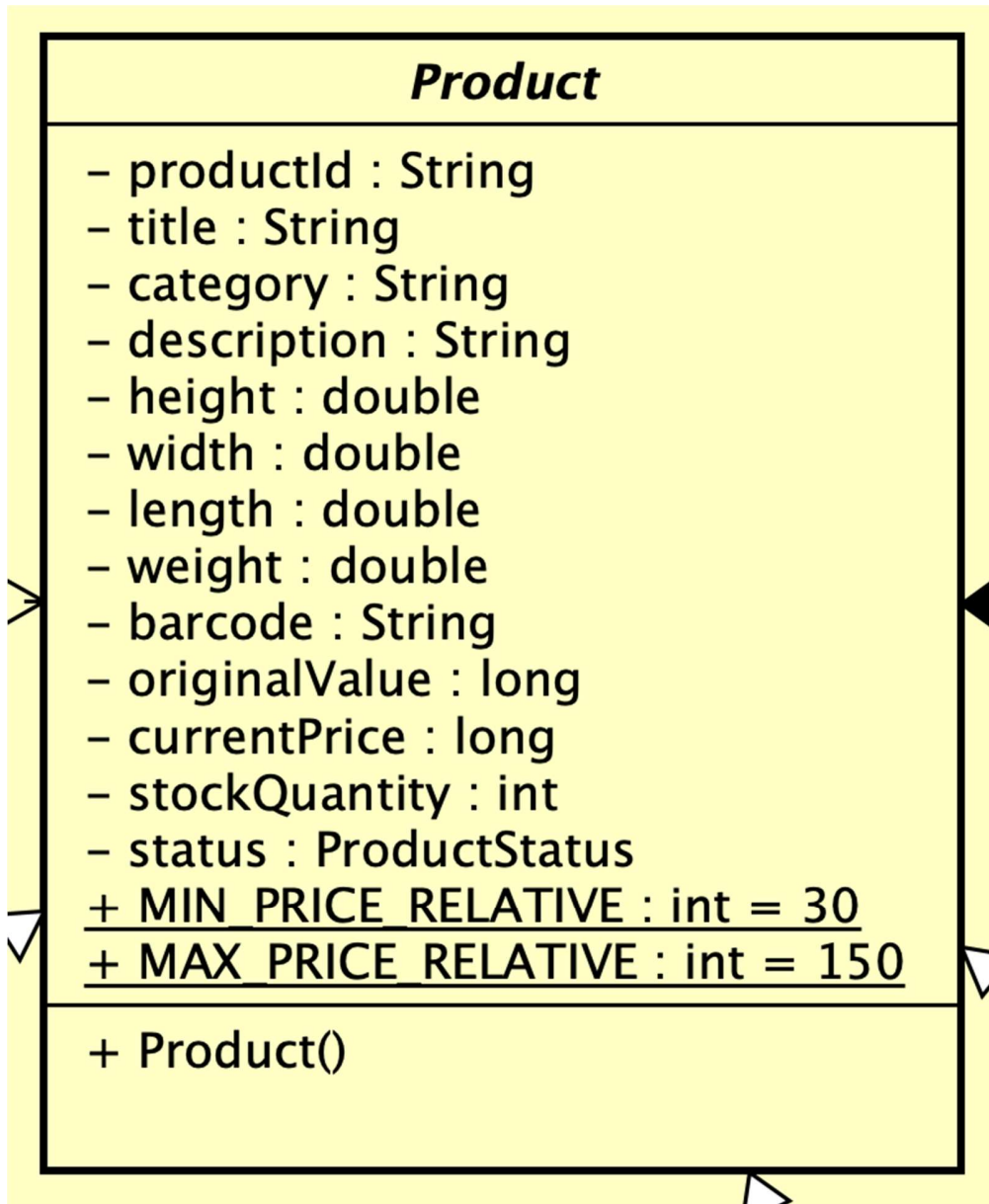


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productid	String		Unique ID of the product
2	title	String		Title of the product

3	category	String		Category of the product
4	description	String		Product description
5	height	double		height
6	width	double		width
7	length	double		length
8	weight	double		weight
9	barcode	String		Unique barcode of the product
10	originalValue	long		Product original value
11	currentPrice	long		Current price of product
12	stockQuantity	int		Product quantity in stock
13	status	ProductStatus		Current status
14	MIN PRICE RELATIVE	int	30	Min current price
15	MAX PRICE RELATIVE	int	150	Max current price

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	Product		Constructor of the product entity

Design for class “PrintableProduct”

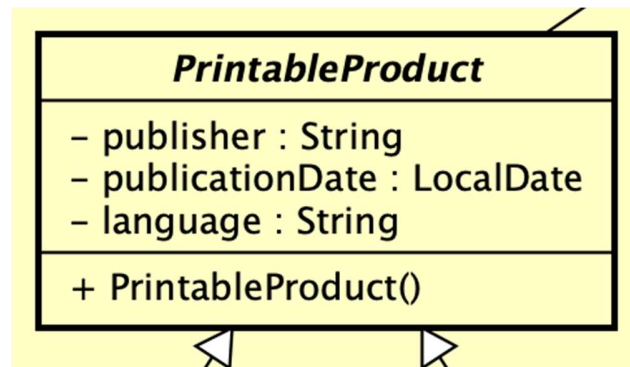


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	publisher	String		Publisher name
2	publicationDate	LocalDate		Publication date
3	language	String		Language written in the product

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	PrintableProduct		Constructor of printable product (Book, Newspaper) entity

Design for class “Book”

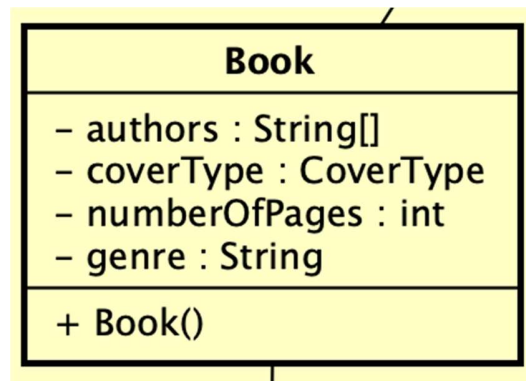


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	authors	String[]		Name of author(s)
2	coverType	CoverType		Type of cover (Paper back, Hard cover)
3	numberOfPages	int		Number of pages
4	genre	String		Genre of product

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	Book		Constructor of Book entity

Design for class “Newspaper”

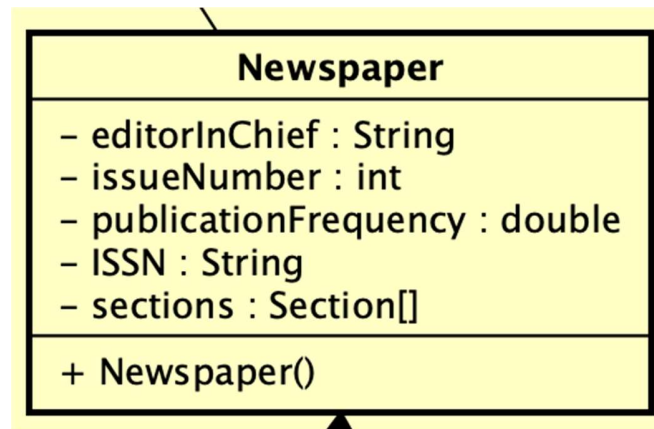


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	editorInChief	String		Name of Chief editor
2	issueNumber	int		Issue number
3	publicationFrequency	double		Publication frequency
4	ISSN	String		ISSN identification
5	sections	Section[]		Sections included in the newspaper

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	Newspaper		Constructor of Newspaper entity

Design for class “Section”

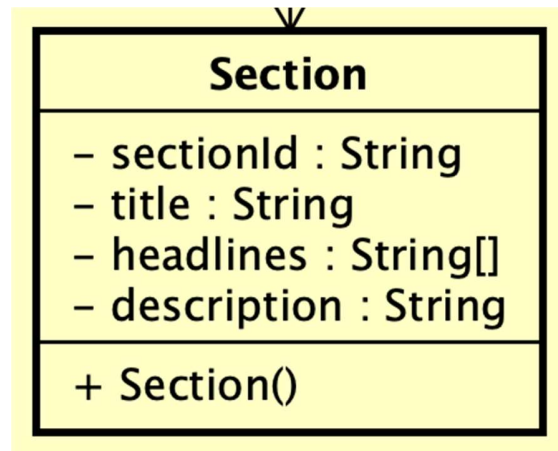


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	sectionId	String		Unique id of section in the newspaper
2	title	String		Title of section
3	headlines	String[]		Headlines
4	description	String		Section description

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	Section		Constructor of section entity

Design for class “CD”

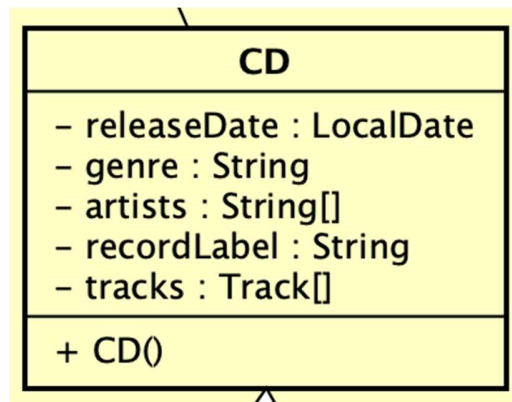


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	releaseDate	LocalDate		Release date
2	genre	String		Genre
3	artists	String[]		Artist(s) in the CD
4	recordLabel	String		Record label
5	tracks	Track[]		Track(s) in the CD

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	CD		Constructor of CD entity

Design for class “Track”

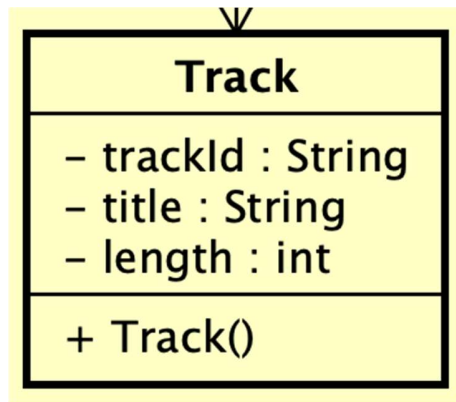


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	trackId	String		Unique ID of the track
2	title	String		Track title
3	length	int		track duration

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	Track		Constructor of Track entity

Design for class “DVD”

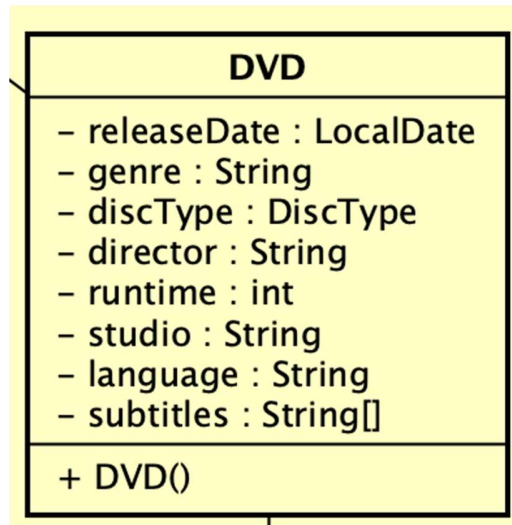


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	releaseDate	LocalDate		Release date
2	genre	String		Genre
3	discType	DiscType		Type of disc (Bluray, HD DVD)
4	director	String		Director name
5	runtime	int		Duration
6	studio	String		Studio
7	language	String		Language
8	subtitles	String[]		Subtitles

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	DVD		Constructor of DVD entity

Design for interface “ProductRepository”

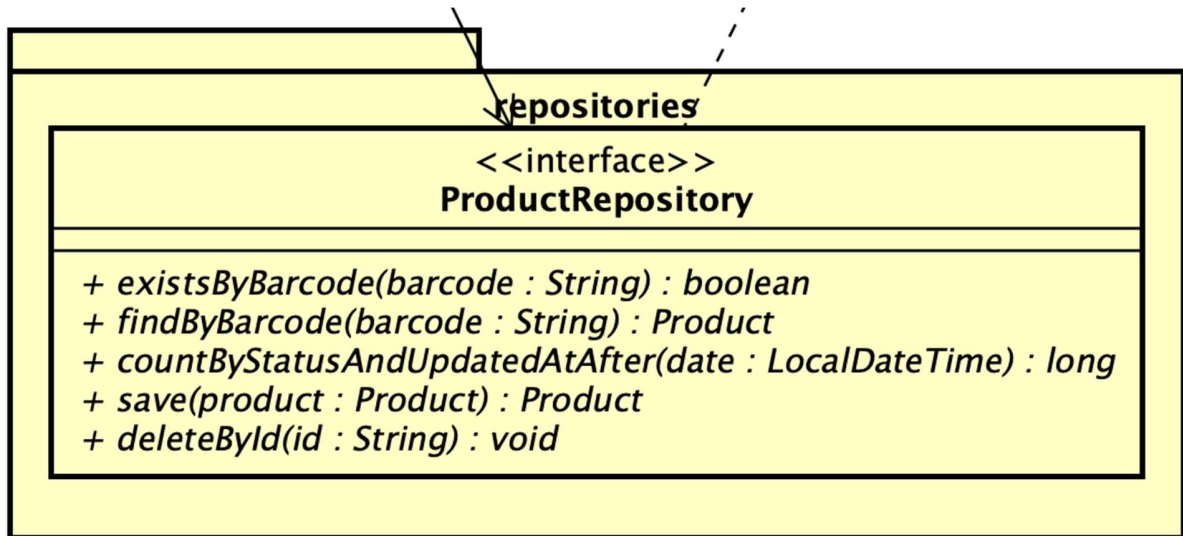


Table 1. Attribute design

#	Name	Data type	Default value	Description
1				

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	existsByBarcode	boolean	Verify if any product with the barcode exists
2	findByBarcode	Product	Search product
3	countByStatusAndUpdatedAtAfter	long	Count number of deletion quota in the day
4	save	Product	Save a new/updated product into database
5	deleteById	void	Delete a product

1. *existsByBarcode(barcode : String) : boolean*

- Parameter

- o barcode: The barcode to be checked if any product with the same barcode exists

- Method

- o Executes a query to determine if a specific barcode already exists

2. *findByBarcode(barcode : String) : Product*

- Parameter
 - barcode: Barcode of the product to search
- Method
 - Retrieves a complete Product entity from the persistence store using its unique barcode

3. *countByStatusAndUpdatedAtAfter(date : LocalDateTime) : long*

- Parameter
 - date: The current date
- Method
 - Returns the count of records updated to a "deleted" or "deactivated" status after the specified timestamp for quota tracking

4. *save(product : Product) : Product*

- Parameter
 - product: The product entity to be saved
- Method
 - Persists a new product or merges changes of an existing product into the database

5. *deleteById(id : String) : void*

- Parameter
 - id: ID of the product to be deleted
- Method
 - Performs a physical removal of the product record from the database using its primary identifier.

Design for class “ProductManagementService”

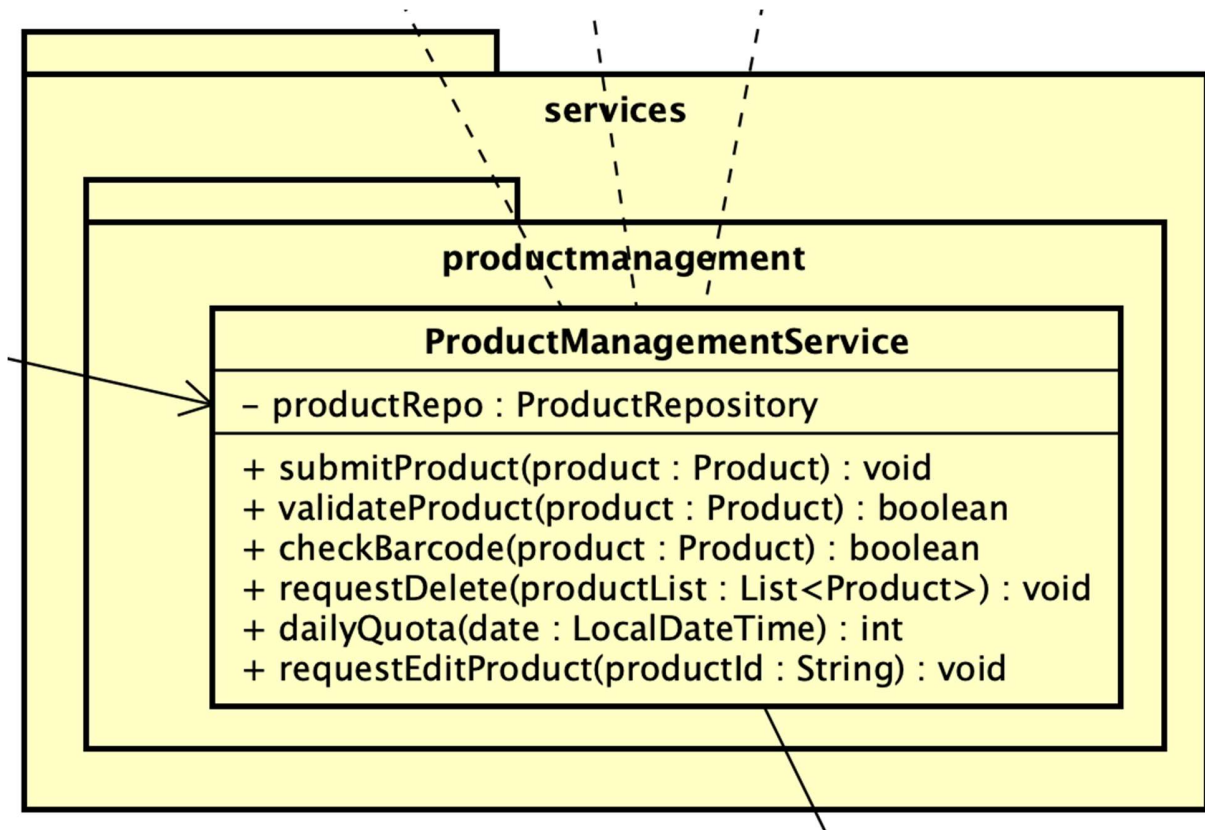


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productRepo	ProductRepository		Product Repository

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	submitProduct	void	Save the product record
2	validateProduct	boolean	Validate the product
3	checkBarcode	boolean	Check if any product with the barcode already existed
4	requestDelete	void	Request to delete products
5	dailyQuota	int	Retrieve the deletion quota of the day
6	requestEditProduct	void	Request to edit product

1. *submitProduct(product : Product) : void*
 - Parameter
 - product: Product entity to be saved
 - Method
 - Coordinates the final submission of a product to the persistence layer after all internal validations are cleared
2. *validateProduct(product : Product) : Boolean*
 - Parameter
 - product: Product entity that its fields need to be verified
 - Method
 - Checks if the product data complies with business constraints
3. *checkBarcode(product : Product) : boolean*
 - Parameter
 - product: Product entity that needto check if its barcode is duplicated with another one existed
 - Method
 - Verifies the uniqueness of the product barcode
4. *requestDelete(productList : List<Product>) : void*
 - Parameter
 - productList: List of products to be deleted
 - Method
 - Evaluates the deletion request by checking stock levels to decide between a hard delete or deactivation.
5. *dailyQuota(date : LocalDateTime) : int*
 - Parameter
 - date: Current date that need to check deletion quota
 - Method
 - Calculates the total number of products deleted on a specific date
6. *requestEditProduct(productId : String) : void*
 - Parameter
 - productid: ID of product to be edited
 - Method
 - Prepares the product data for editing by retrieving the current state

Design for class “CreateProductDTO”

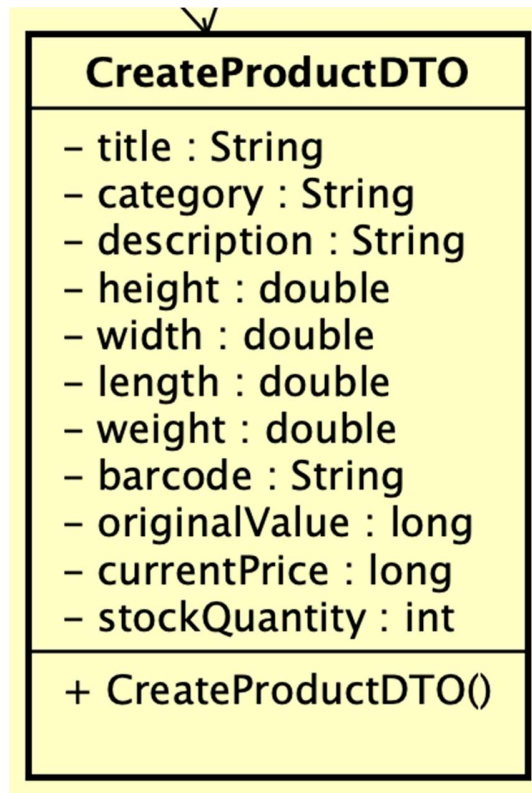


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	title	String		Title of the product
2	category	String		Category of the product
3	description	String		Product description
4	height	double		height
5	width	double		width
6	length	double		length
7	weight	double		weight
8	barcode	String		Unique barcode of the product
9	originalValue	long		Product original value

10	currentPrice	long		Current price of product
11	stockQuantity	int		Product quantity in stock

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	CreateProductDTO		Constructor of CreateProductDTO entity

Design for class “UpdateProductDTO”

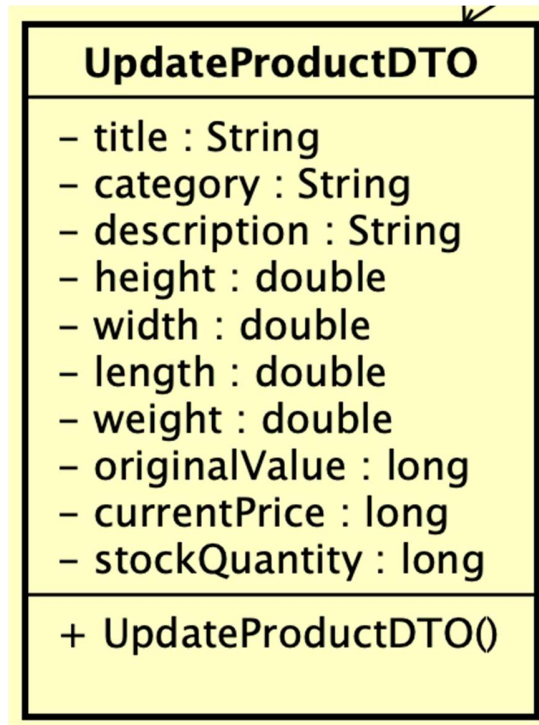


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	title	String		Title of the product
2	category	String		Category of the product
3	description	String		Product description
4	height	double		height
5	width	double		width
6	length	double		length
7	weight	double		weight
8	barcode	String		Unique barcode of the product
9	originalValue	long		Product original value
10	currentPrice	long		Current price of product

11	stockQuantity	int		Product quantity in stock
----	---------------	-----	--	------------------------------

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	UpdateProductDTO		Constructor of UpdateProductDTO entity

Design for class “DeleteProductDTO”



Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productIds	List<String>		IDs of products to be deleted

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	DeleteProductDTO		Constructor of DeleteProductDTO entity

Design for class “CartItemRequest”

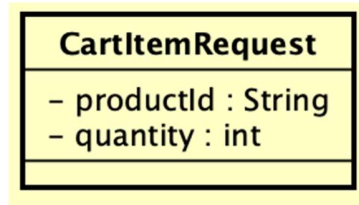


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productId	String		Unique ID of product
2	quantity	int		Quantity of product in cart

Table 2. Operation design

#	Name	Return type	Description (purpose)
1			

Design for class “CartItemReponse”

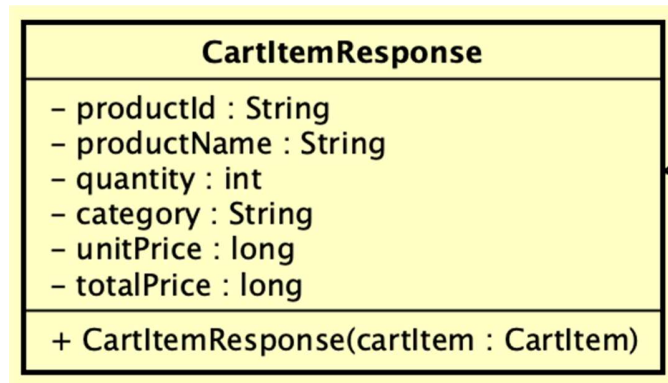


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productId	String		Unique ID of product
2	productName	String		Product name
3	quantity	int		Quantity of product in cart
4	category	String		Product category
5	unitPrice	long		Price of a product unit
6	totalPrice	long		Price of a product * quantity

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	CartItemResponse		Constructor of CartItemResponse entity

Design for class “CartReponse”

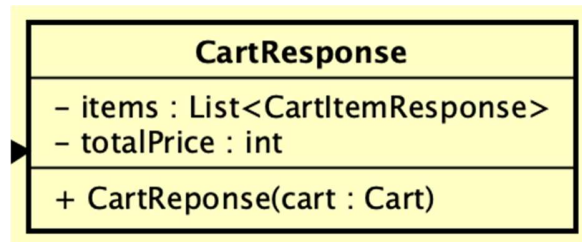


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	items	List<CartItemResponse>		List of items in cart
2	totalPrice	int		Total price of products

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	CartResponse		Constructor of CartResponse entity

Design for class “DeliveryRequest”

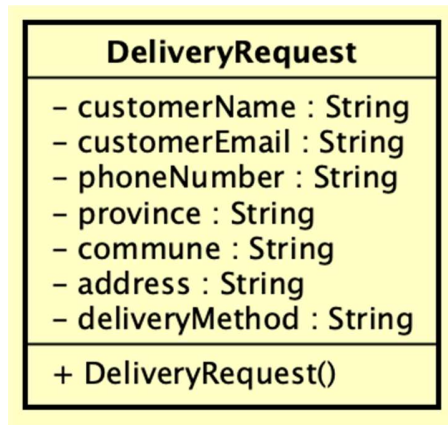


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	customerName	String		Customer name
2	customerEmail	String		Customer email
3	phoneNumber	String		Phone number
4	province	String		Province
5	commune	String		Commune
6	address	String		Detail address
7	deliveryMethod	String		Delivery method

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	DeliveryRequest		Constructor of DeliveryRequest entity

Design for class “OrderItemResponse”

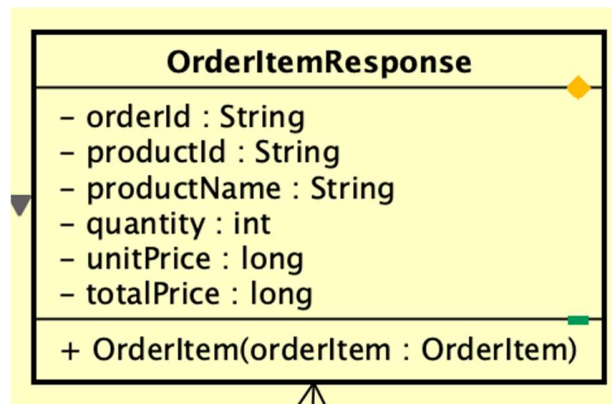


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderId	String		Order ID
2	productId	String		Product ID
3	productName	String		Product name
4	quantity	int		Quantity of product
5	unitPrice	long		Price of a product unit
6	totalPrice	long		Price of a product * quantity

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	OrderItemReponse		Constructor of OrderItemResponse entity

Design for class “InvoiceResponse”

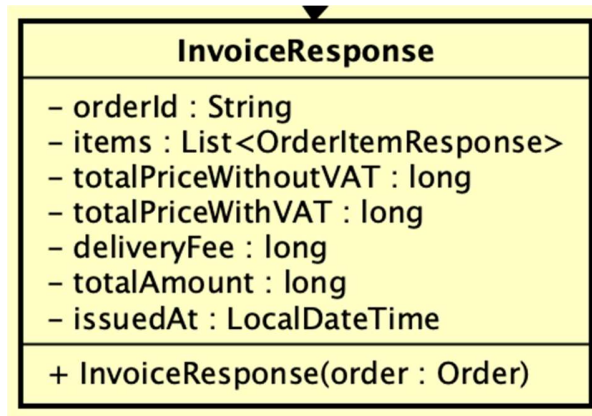


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderId	String		Order ID
2	items	List<OrderItemResponse>		List of items
3	totalPriceWithoutVAT	long		Total price without VAT
4	totalPriceWithVAT	long		Total price with VAT
5	deliveryFee	long		Delivery fee
6	totalAmount	long		Total product amount
7	issuedAt	LocalDateTime		Time issued

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	InvoiceResponse		Constructor of InvoiceResponse entity

Design for class “ProductController”

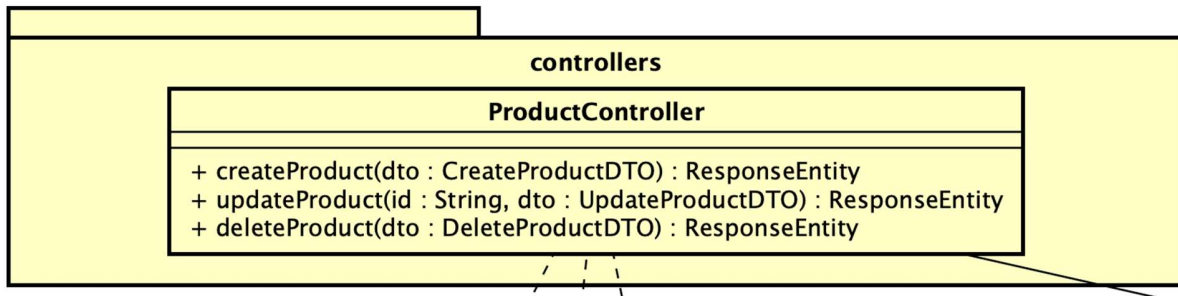


Table 1. Attribute design

#	Name	Data type	Default value	Description
1				

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	createProduct	ResponseEntity	
2	updateProduct	ResponseEntity	
3	deleteProduct	ResponseEntity	

1. *createProduct(dto : CreateProductDTO) : ResponseEntity*

- Parameter

- dto: The CreateProductDTO containing the information of the new product

- Method

- Receives product data
- Triggers the creation process in the service layer
- returns a status upon success

2. *updateProduct(id : String, dto : UpdateProductDTO) : ResponseEntity*

- Parameter

- id: Id of the updated product
- dto: The UpdateProductDTO containing the edited information of the product

- Method

- Receives an existing product's ID and updated data
- Coordinates the modification process
- Returns the updated product status

3. *deleteProduct(dto : DeleteProductDTO) : ResponseEntity*

- Parameter

- dto: The DeleteProductDTO containing the list of product IDs to be deleted
- Method
 - Processes a request to remove or deactivate products based on the provided list of identifiers.

Design for interface “OrderRepository”

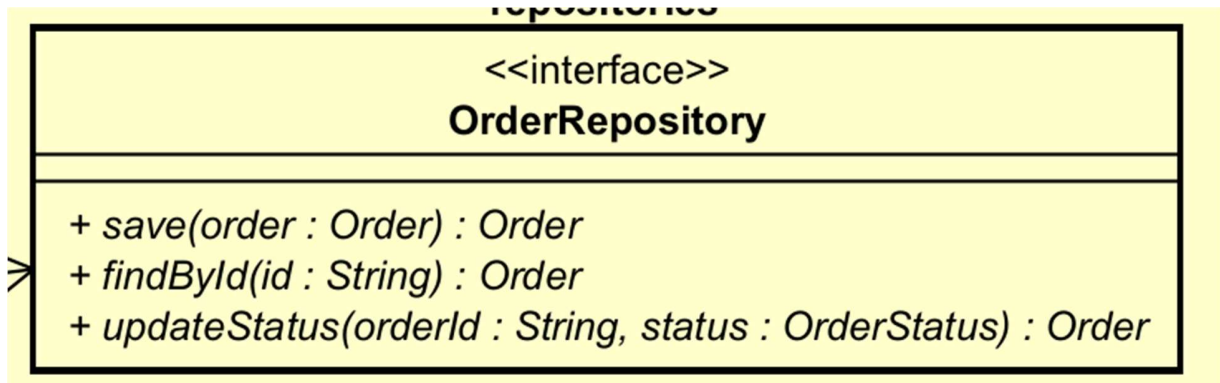


Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	save	Order	Save the order to the database, return the order created
2	findById	Order	Find an order by the ID, return the order found
3	updateStatus	Order	Update the status of the order with corresponding ID, return the updated order

Parameter:

- *save(order : Order) : Order*
 - order: the Order object to be pushed to database
- *findById(id : String) : Order*
 - id: the ID to query
- *updateStatus(orderId : String, status : OrderStatus) : Order*
 - orderId: the ID to query
 - status: the status of the order to be updated

Design for interface “PaymentTransactionRepository”

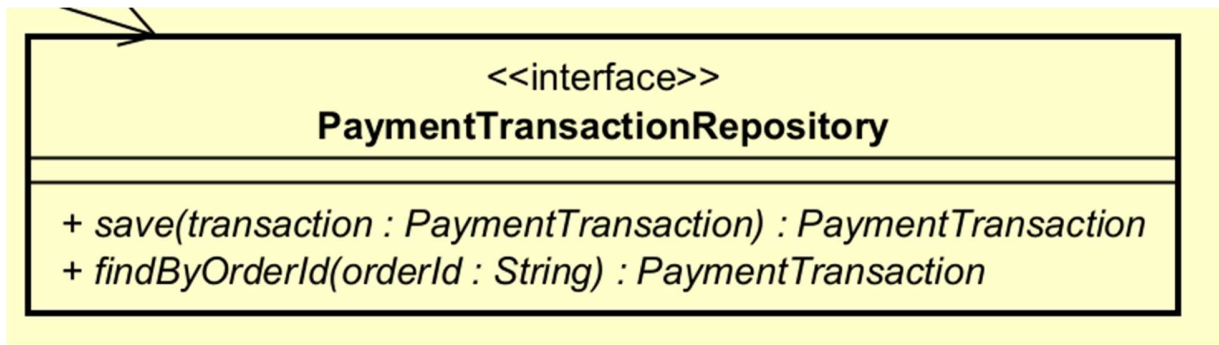


Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	save	PaymentTransaction	Save the payment transaction entity to database, return the saved transaction entity
2	findById	PaymentTransaction	Find the payment transaction associated with an entity

Parameter:

- *save(transaction : PaymentTransaction) : PaymentTransaction*
 - o transaction: the PaymentTransaction object to be pushed to database
- *findById(orderId : String) : PaymentTransaction*
 - o orderId: the order ID to query

Design for class “OrderService”

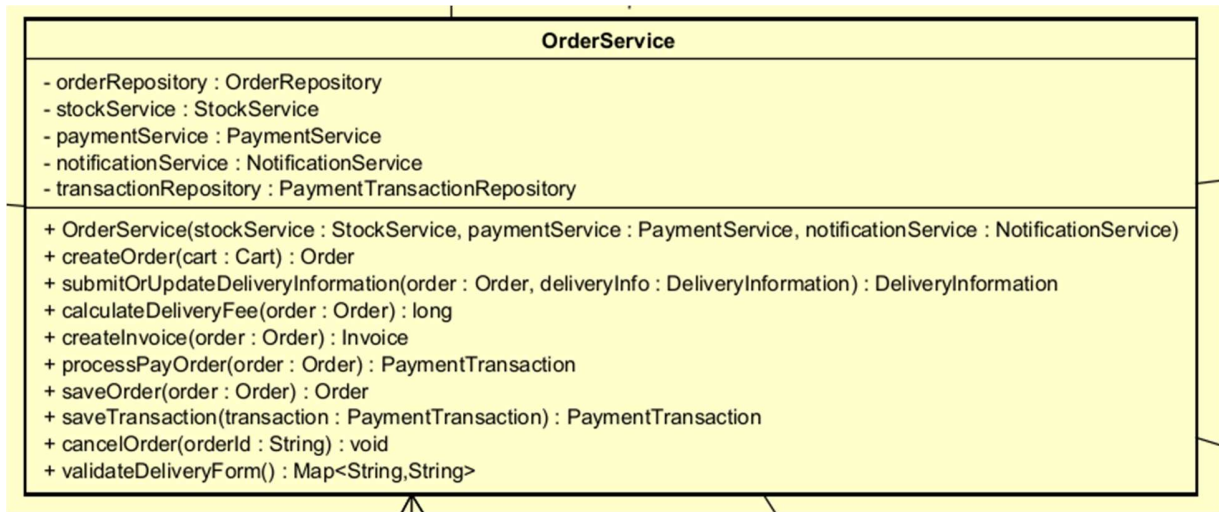


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderRepository	OrderRepository		Spring Data JPA repository for Order persistence
2	stockService	StockService		Service for stock
3	paymentService	PaymentService		Service for payment
4	notificationService	NotificationService		Interface; service for email notification
5	transactionRepository	TransactionRepository		Spring Data JPA repository for PaymentTransaction persistence

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	OrderService		Constructor
2	createOrder	Order	Create a new Order instance based on cart content
3	submitOrUpdateDeliveryInformation	DeliveryInformation	Set or update the delivery information

4	calculateDeliveryFee	long	Calculate the delivery fee for an order and return it
5	createInvoice	Invoice	Generate invoice with totals including VAT and delivery fee
6	processPayOrder	PaymentTransaction	Process payment for the order via selected payment method
7	saveOrder	Order	Persist the order entity to the database
8	saveTransaction	PaymentTransaction	Persist payment transaction details to the database
9	cancelOrder	void	Cancel an order
10	validateDeliveryForm	Map<String,String>	Validate delivery information fields and return a map from invalid field names to corresponding problems

1. OrderService(stockService, paymentService, notificationService)

Parameter

- stockService: StockService instance for stock management operations
- paymentService: PaymentService instance for payment processing
- notificationService: NotificationService instance for sending notifications

Method

- Initializes all service dependencies via constructor injection
- Sets orderRepository and transactionRepository for database operations

2. createOrder(cart: Cart)

Parameter

- cart: Cart object containing items to be ordered

Exception

- EmptyCartException if cart is empty when creating order

Method

- Validates cart is not empty (throws EmptyCartException if empty)
- Checks stock availability for all items using stockService
- Creates new Order entity with status Draft
- Calculates subtotal from cart items
- Returns the created Order entity

3. submitOrUpdateDeliveryInformation(order, deliveryInfo)

Parameter

- order: Order entity to update
- deliveryInfo: DeliveryInformation object with customer address details

Exception

- InvalidDeliveryInfoException if delivery information contains invalid or missing fields

Method

- Validates deliveryInfo fields using validateDeliveryForm()
- Throws InvalidDeliveryInfoException if validation fails
- Sets delivery information on the order entity
- Returns the updated DeliveryInformation

4. calculateDeliveryFee(order: Order)

Parameter

- order: Order entity containing items and delivery location

Method

- Calculates total weight of all items in order
- Determines destination province from delivery information
- Applies shipping rules:
 - o Free shipping if order total > 100,000 VND (max 25,000 VND discount)

- Hanoi/HCMC: 22,000 VND for first 3kg
- Other provinces: 30,000 VND for first 0.5kg
- Additional 2,500 VND per 0.5kg
- Returns calculated fee in VND

5. createInvoice(order: Order)

Parameter

- order: Order entity to generate invoice for

Method

- Calculates total price without VAT (sum of item prices)
- Applies 10% VAT to get total price with VAT
- Adds delivery fee to get total amount
- Creates Invoice entity with all calculated values
- Sets issuedAt timestamp to current datetime
- Returns the Invoice entity

6. processPayOrder(order: Order)

Parameter

- order (in processPayOrder): Order entity with payment details

Method

- Calls paymentService to process payment based on selected method
- Creates PaymentTransaction entity with transaction details
- Updates order status to "Pending"
- Calls notificationService to send confirmation email
- Returns the PaymentTransaction entity

7. saveOrder(order: Order)

Parameter

- order (in saveOrder): Order entity to persist

Method

- Persists order entity using orderRepository.save()
- Returns the saved Order with generated ID

8. saveTransaction(transaction: PaymentTransaction)

Parameter

- transaction (in saveTransaction): PaymentTransaction entity to persist

Method

- Persists transaction using transactionRepository.save()
- Returns the saved PaymentTransaction

9. cancelOrder(orderId: String)**Parameter**

- orderId (in cancelOrder): String identifier of the order to cancel

Method

- Retrieves order by ID
- Updates order status to Cancelled
- Calls paymentService to initiate refund
- Sends cancellation notification via notificationService
- Saves updated order

10. validateDeliveryForm()**Method**

- Checks all required fields: customerName, phone, email, province, address
- Validates phone format (10 digits, starts with 0)
- Validates email format (contains @)
- Returns Map of field names to error messages (empty if valid)

Design for class “StockService”

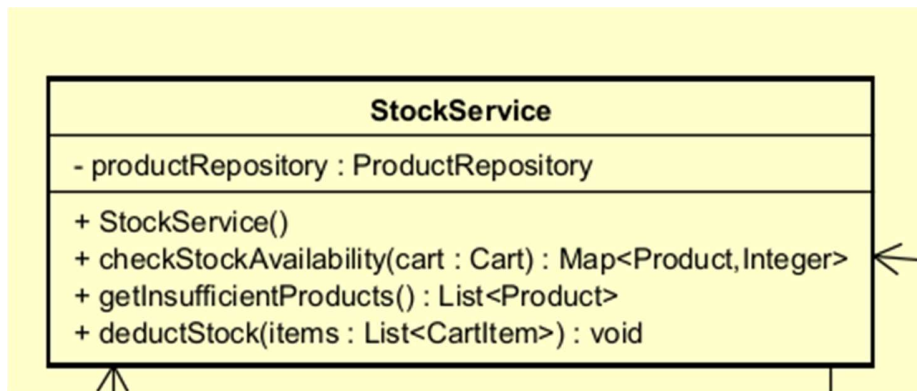


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productRepository	ProductRepository		Spring Data JPA repository for Product persistence

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	StockService		Constructor
2	checkStockAvailability	Map<Product, Integer>	Validates stock levels for all items in cart and returns insufficient quantities
3	getInsufficientProducts	List<Product>	Returns list of products that have insufficient stock for the current order
4	deductStock	void	Decreases stock quantities for products after successful order placement

1. StockService()

Method

2. checkStockAvailability(cart: Cart): Map<Product, Integer>

Parameter

- cart (in checkStockAvailability): Cart object containing list of CartItems with requested quantities

Method

- Iterates through all CartItems in the cart

- For each item, retrieves the corresponding Product from database via productRepository
- Compares requested quantity with product.stockQuantity
- If stock is insufficient, adds entry to map with Product as key and (requestedQuantity - availableQuantity) as value
- Returns empty map if all items have sufficient stock
- Returns map with only insufficient products and their shortage amounts

3. getInsufficientProducts(): List<Product>

Method

- Returns list of Product objects that failed stock validation
- Used to display to customer which products need quantity adjustment
- Called after checkStockAvailability() to get detailed product information

4. deductStock(items: List<CartItem>): void

Parameter

- items: List of CartItem objects representing products and quantities to deduct from stock

Method

- Iterates through each CartItem in the list
- Retrieves current Product from database
- Decreases product.stockQuantity by item.quantity
- Validates that resulting stock is not negative
- Saves updated Product entity back to database via productRepository
- Records stock adjustment in audit log (for manual adjustments)
- Called after successful payment to permanently reduce inventory

Design for class “CartService”

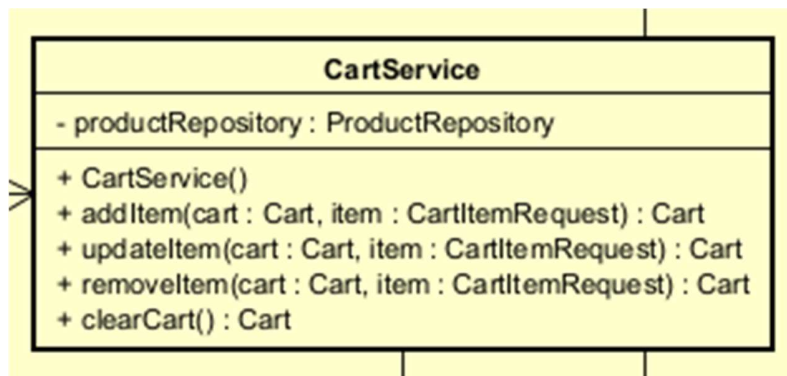


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productRepository	ProductRepository		Spring Data JPA repository for Product persistence

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	CartService		Constructor
2	addItem	Cart	Adds a new item or updates quantity of existing item in the cart
3	updateItem	Cart	Updates the quantity of an existing item in the cart
4	removeItem	Cart	Removes a specific item from the cart
5	clearCart	Cart	Removes all items from the cart, returning an empty cart

Parameter (common to addItem, updateItem, removeItem)

- cart: Cart object representing the current shopping cart session
- item: CartItemRequest object containing productId and quantity information

1. CartService()

Method

2. addItem(cart : Cart, item : CartItemRequest) : Cart

Exception

- ProductNotFoundException if product does not exist (either deactivated or deleted)
- NotEnoughStockException if product.stockQuantity >= item.quantity

Method

- Validates that item.quantity > 0
- Uses productRepository to find product by item.productId
- Throws ProductNotFoundException if product does not exist
- Checks if product.stockQuantity >= item.quantity, throws NotEnoughStockException if not
- Checks if item already exists in cart.productList
 - o If exists: updates the quantity of existing CartItem
 - o If not exists: creates new CartItem with product and quantity, adds to cart.productList
- Recalculates cart.totalPrice by summing (item.quantity × item.product.currentPrice) for all items
- Returns the updated cart object

3. updateItem(cart : Cart, item : CartItemRequest) : Cart

Exception

- ProductNotFoundException if product does not exist (either deactivated or deleted)

Method

- Validates that item.quantity > 0
- Finds the CartItem in cart.productList with matching productId
- Throws ProductNotFoundException if item not found in cart
- Uses productRepository to verify product still exists and check current stock
- Validates that requested quantity <= product.stockQuantity
- Updates the CartItem.quantity to the new value
- Recalculates cart.totalPrice
- Returns the updated cart object

4. removeItem(cart : Cart, item : CartItemRequest) : Cart

Method

- Finds the CartItem in cart.productList with matching productId
- If found, removes the CartItem from cart.productList

- Recalculates cart.totalPrice
- Returns the updated cart object
- If item not found, returns cart unchanged

5. clearCart() : Cart

Method

- Creates a new empty Cart instance or clears existing cart.productList
- Sets cart.totalPrice to 0
- Returns the empty cart object

Design for class “CartController”

CartController	
- stockService : StockService - cartService : CartService	
+ CartController(stockService : StockService, cartService : CartService) + getCart(httpSession : HttpSession) : ResponseEntity<CartResponse> + addItem(httpSession : HttpSession, item : CartItemRequest) : ResponseEntity<CartResponse> + updateItem(httpSession : HttpSession, item : CartItemRequest) : ResponseEntity<CartResponse> + removeItem(httpSession : HttpSession, item : CartItemRequest) : ResponseEntity<CartResponse> + checkStockAvailability(httpSession : HttpSession) : ResponseEntity<Map<String,Integer>> + clearCart(httpSession : HttpSession) : ResponseEntity<Void> - getOrCreateCart(httpSession : HttpSession) : Cart	

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	stockService	StockService		Service dependency injected for validating product stock availability during cart operations
2	cartService	CartService		Service dependency injected for handling cart business logic (add, update, remove, clear items)

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	CartController		Constructor
2	getCart	ResponseEntity<CartResponse>	Retrieves the current shopping cart from the HTTP session and returns it as a structured response
3	addItem	ResponseEntity<CartResponse>	Adds a new product to the cart or increments quantity if it already exists
4	updateItem	ResponseEntity<CartResponse>	Modifies the quantity of a specific product already present in the cart
5	removeItem	ResponseEntity<CartResponse>	Deletes a specific product entry from the cart entirely
6	checkStockAvailability	ResponseEntity<Map<String,Integer>>	Validates inventory levels for a given cart and returns a map of

			insufficient products with shortage quantities
7	clearCart	ResponseEntity<Void>	Empties the entire shopping cart by removing it from the session storage
8	getOrCreateCart	Cart	Private helper method that retrieves an existing cart from session or initializes a new one

Parameter (common to almost all methods):

- httpSession: Current HTTP session context where the Cart object is stored as an attribute ("cart").
- item: CartItemRequest object containing productId and quantity information

1. CartController(stockService : StockService, cartService : CartService)

Parameter

- stockService: Instance of StockService used to delegate stock validation calls.
- cartService: Instance of CartService used to delegate cart manipulation logic.

2. getCart(httpSession : HttpSession) : ResponseEntity<CartResponse>

Method

- Calls getOrCreateCart(httpSession) to fetch the session cart.
- Maps the Cart entity to CartResponse DTO and returns ResponseEntity.ok(response).

3. addItem(httpSession : HttpSession, item : CartItemRequest) : ResponseEntity<CartResponse>

Method

- Retrieves cart via getOrCreateCart().
- Delegates to cartService.addItem(cart, item) which validates product existence and stock.
- Updates the session attribute "cart" with the modified cart. Maps to CartResponse and returns HTTP 200.

4. updateItem(httpSession : HttpSession, item : CartItemRequest) : ResponseEntity<CartResponse>

Method

- Retrieves cart from session.
- Delegates to cartService.updateItem(cart, item) to adjust quantity.
- Updates session and returns CartResponse with HTTP 200.

5. removeItem(httpSession : HttpSession, item : CartItemRequest) : ResponseEntity<CartResponse>

Method

- Retrieves cart from session.
- Delegates to cartService.removeItem(cart, item) to delete the specified product entry.
- Updates session and returns CartResponse with HTTP 200.

6. checkStockAvailability(httpSession : HttpSession) : ResponseEntity<Map<String,Integer>>

Method

- Retrieves cart from session.
- Delegates to stockService.checkStockAvailability(cart).
- Returns a Map<String, Integer> where keys are productIds and values are missing quantities.
- Wraps result in ResponseEntity with HTTP 200.

7. clearCart(httpSession : HttpSession) : ResponseEntity<Void>

Method

- Retrieves cart from session and delegates to stockService.clearCart() to clear cart

8. getOrCreateCart(httpSession : HttpSession) : Cart

Method

- Retrieves cart from session.

Design for class “OrderController”

OrderController	
- orderService : OrderService	
+ OrderController(orderService : OrderService) + placeOrder(httpSession : HttpSession) : ResponseEntity<Void> + getOrderDetail() : ResponseEntity<InvoiceResponse> + cancelOrder(orderId : String) : ResponseEntity<Void> + getInvoice(orderId : String) : ResponseEntity<InvoiceResponse>	

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderService	OrderService		Service dependency injected for handling order creation, cancellation, and retrieval logic.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	OrderController		Constructor
2	placeOrder	ResponseEntity<Void>	Transitions the cart in the session to a pending Order and stores it in the session.
3	getOrderDetail	ResponseEntity<InvoiceResponse>	Retrieves the details of the current order being processed in the session.
4	cancelOrder	ResponseEntity<Void>	Cancels an existing order based on the provided Order ID.
5	getInvoice	ResponseEntity<InvoiceResponse>	Retrieves the invoice details for a specific Order ID (used for

			success screen/history).
--	--	--	-----------------------------

1. OrderController(orderService)

2. placeOrder(httpSession):

Method

- Retrieves the Cart object from httpSession
- Validates that the cart is not empty.
- Calls orderService.createOrder(cart) to validate stock and create a new Order entity with status Draft.
- Stores the created Order in httpSession under the attribute key "currentOrder".
- Returns ResponseEntity.ok() (HTTP 200).

3. getOrderDetail():

Method

- Retrieves the Order object from httpSession (attribute "currentOrder").
- If the order is null, throws an exception or returns 404 (though typically this is called after placeOrder).
- Converts the Order entity into an InvoiceResponse DTO (calculating VAT and totals if not already done)
- Returns ResponseEntity.ok(invoiceResponse).

4. cancelOrder(orderId):

Method

- Calls orderService.cancelOrder(orderId).
- The service handles logic such as checking if the order is in a cancellable state (e.g., PENDING) and initiating refunds.
- Returns ResponseEntity.ok() (HTTP 200).

5. getInvoice(orderId):

Method

- Calls orderService to retrieve the Order entity by orderId.
- Converts the Order entity into an InvoiceResponse DTO.
- Returns ResponseEntity.ok(invoiceResponse).

Design for class “NotificationService”

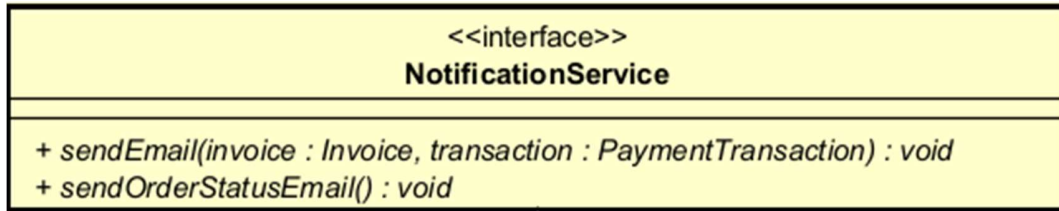


Table 2. Operation design

#	Name	Return type	Description (purpose)
1	sendEmail	void	Send an email notification containing invoice and payment transaction information to the customer.
2	sendOrderStatusEmail	void	Send an email notification to inform the customer about the current order status.

1.sendEmail(invoice : Invoice, transaction : PaymentTransaction) : void

invoice: invoice object containing order and payment details.

transaction: payment transaction object containing transaction result information.

**sendEmail(invoice : Invoice, transaction :
PaymentTransaction) : void**

invoice: invoice object containing order and payment details.

transaction: payment transaction object containing transaction result information.

Design for class “DeliveryController”

DeliveryController	
- orderService : OrderService	
+ DeliveryController(orderService : OrderService) + submitDeliveryInformation(request : DeliveryRequest) : ResponseEntity<DeliveryInformation> + getDeliveryFee() : ResponseEntity<Long> + validateDeliveryForm() : ResponseEntity<Map<String,String>>	

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderService	OrderService		Service dependency injected for handling order creation, cancellation, and retrieval logic.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	DeliveryController		Constructor

2	submitDeliveryInformation	ResponseEntity<DeliveryInformation>	Accepts delivery details from the client, validates them, and updates the current order in the session.
3	getDeliveryFee	ResponseEntity<Long>	Calculates and returns the shipping fee for the current order based on weight and destination.
4	validateDeliveryForm	ResponseEntity<Map<String,String>>	Validates the format and completeness of the delivery information fields.

1. DeliveryController(orderService)

2. submitDeliveryInformation(request, httpSession):

Method

- Retrieves the Order object from httpSession (attribute "currentOrder").
- Calls validateDeliveryForm() logic to check input validity. If invalid, returns 400 Bad Request with error map.
- Calls orderService.submitOrUpdateDeliveryInformation(order, request).
- Updates the Order in the session with the new delivery info.
- Returns ResponseEntity.ok(updatedDeliveryInformation).

3. getDeliveryFee(httpSession):

Method

- Retrieves the Order object from httpSession.
- Calls orderService.calculateDeliveryFee(order).
- Returns ResponseEntity.ok(feeAmount).

4. validateDeliveryForm(request):

Method

- Checks if customerName, phone, email, province, address are not null/empty.
- Validates phone format (must start with '0', 10 digits).

- Validates email format (must contain '@').
- Populates a Map<String, String> with field names as keys and error messages as values for any invalid fields.
- Returns ResponseEntity.ok(errorMap) (empty map if valid).

Design for class “PaymentController”

PaymentController	
- paymentService : PaymentService	
+ payByCreditCard(request : PaymentRequest) : ResponseEntity<PaymentResponse> + getPaymentTransaction(orderId : String) : ResponseEntity<?> + getTransactionFailure(orderId : int) : ResponseEntity<?>	

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	paymentService	PaymentService	null	Reference to the interface handling payment business logic. Injected via constructor.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	payByCreditCard	ResponseEntity	Exposes the API endpoint to receive credit card payment requests from the frontend and returns the payment result.
2	getPaymentTransaction	ResponseEntity<?>	Retrieves the details of a specific payment transaction based on the order ID.
3	getTransactionFailure	ResponseEntity<?>	Retrieves the failure reasons or details for a specific failed transaction.

Detailed Specification for payByCreditCard:

- **Parameter:**

- + `request`: None, Data Transfer Object containing the credit card details and order information sent from the client.

- **Exception:**

- + `IllegalArgumentException` if the request payload is null or missing required fields.

- + `PaymentException` if the underlying service fails to process the payment.

- **Method:**

- + Receives the HTTP POST request.

- + Delegates the business logic by calling `paymentService.processCreditCardPayment(request)`.

- + Wraps the returned `PaymentResponse` into a `ResponseEntity` with an HTTP 200 (OK) status if successful, or HTTP 400/500 if an exception is caught.

Design for class “PaymentRequest”

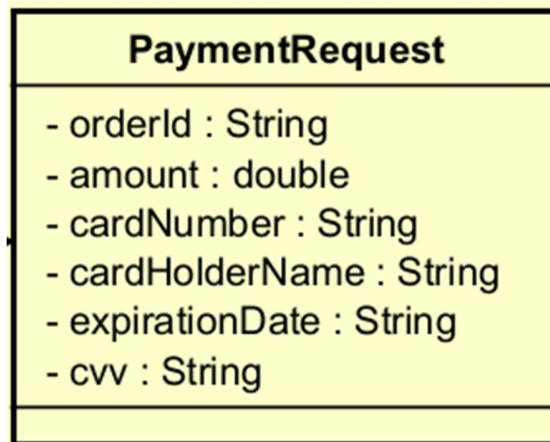


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderId	String	null	Unique identifier for the order being paid.
2	amount	double	0.0	The total amount to be charged to the credit card.
3	cardNumber	String	null	The 16-digit credit card number.
4	cardHolderName	String	null	The name printed on the credit card.
5	expirationDate	String	null	The expiration date of the card (e.g., MM/YY).
6	cvv	String	null	The 3-digit security code on the back of the card.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	getters/setters	Various	Standard accessor and mutator methods for the attributes.

Design for class “CreditCardPaymentService”

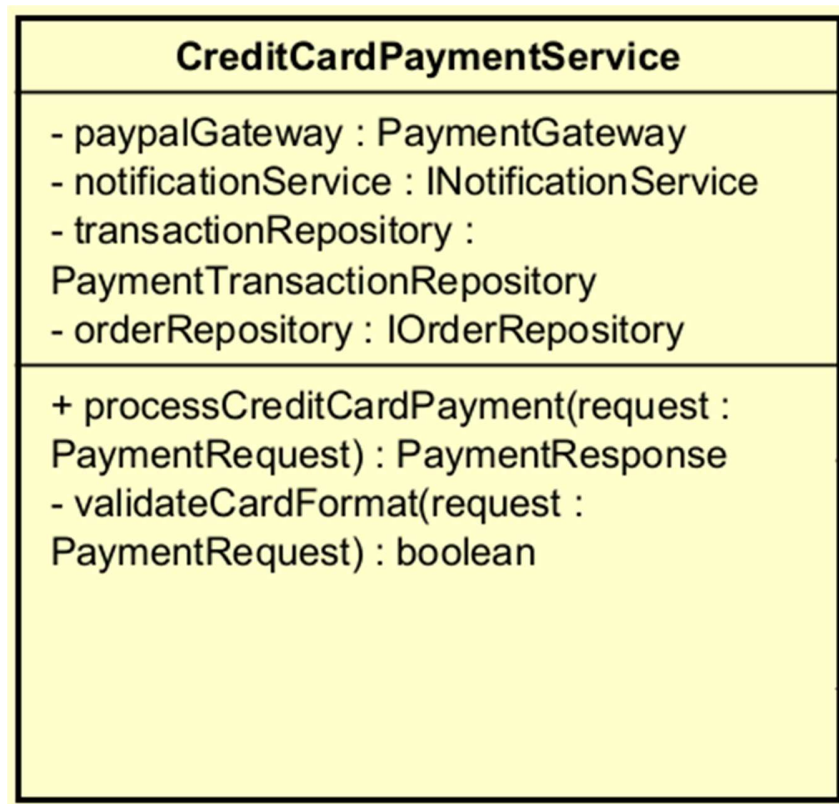


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	paypalGateway	IPaymentGateway	null	Service interface for communicating with the external Paypal API.
2	notificationService	INotificationService	null	Service interface for sending emails to the user.
3	transactionRepository	IPaymentTransactionRepository	null	DAO for persisting transaction

				records to the database.
4	orderRepository	IOrderRepository	null	DAO for fetching and updating the order status.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	processCreditCardPayment	mentPaymentResponse	Orchestrates the entire credit card payment flow: validation, gateway processing, saving state, and notification.
2	validateCardFormat	boolean	A private helper method to ensure the credit card data matches expected formats before sending to the gateway.

Detailed Specification for processCreditCardPayment:

- **Parameter:**
 - + request: None, The DTO containing the payment details.
- **Exception:**
 - + InvalidCardFormatException if validateCardFormat(request) returns false.
 - + GatewayTimeoutException if the paypalGateway does not respond.

- **Method:**

- + Calls `validateCardFormat(request)` to verify the card length and CVV format.

- + Calls `paypalGateway.processPayment(request.getCardNumber(), request.getCvv(), request.getAmount())` to attempt the charge.

- + Creates a new `PaymentTransaction` entity based on the gateway's `TransactionResult`.

- + Calls `transactionRepository.save(transaction)` to store the result in the database.

- + If successful, calls `notificationService.sendEmail(...)` to notify the user.

- + Maps the transaction details into a `PaymentResponse` object and returns it to the controller.

Design for class “VietQRController”

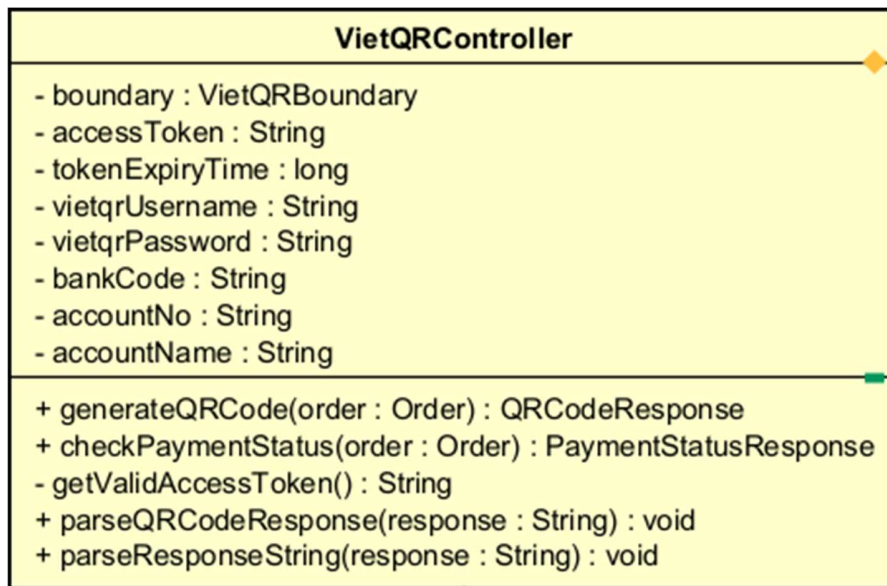


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	boundary	VietQRBoundary		Boundary object used to send HTTP requests to VietQR APIs.
2	accessToken	String		Current VietQR access token used for authenticated API calls.
3	tokenExpiryTime	long		Expiration time of the current access token in milliseconds.
4	vietqrUsername	String		Username of VietQR sandbox
5	vietqrPassword	String		Password of VietQR sandbox
6	bankCode	String		Bank code of the receiver account used to generate QR payment.

7	accountNo	String		Bank account number used to receive customer payment.
8	accountName	String		Bank account owner name

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	generateQRCode	QRCodeResponse	Generate a QR code for the given order and return the QR payment information.
2	checkPaymentStatus	PaymentStatusResponse	Check the payment status of the given order and return the result.
3	getValidAccessToken	String	Return a valid VietQR access token, reusing the current one if it is still valid or requesting a new one if needed.
4	parseQRCodeResponse	void	Parse the raw VietQR QR generation response string and extract QR-related information.
5	parseResponseString	void	Parse the raw VietQR payment status response string and extract payment status information.

Parameter:

- generatePaymentQR(orderId : String) : QRCodeResponse
- orderId: the ID of the order used to generate the payment QR code

- `checkPaymentStatus(orderId : String) : PaymentStatusResponse`
 - `orderId`: the ID of the order used to check the payment status
- `parseQRCodeResponse(response : String) : void`
 - `response`: raw response string returned from VietQR QR code generation API.
- `parseResponseString(response : String) : void`
 - `response`: raw response string returned from VietQR payment status API.

Exception

- `InvalidTokenException`: if VietQR returns an invalid access token.
- `PaymentException`: if QR generation or payment status checking cannot be completed.
- `UnknownException`: if an unexpected error occurs during VietQR API communication or response handling.

Method

parseQRCodeResponse(response : String)

- Reads the raw VietQR QR response string.
- Extracts QR-related fields from the response.
- Stores the extracted values for later use in the returned QR object.

parseResponseString(response : String)

- Reads the raw VietQR payment status response string.
- Extracts status-related fields from the response.
- Stores the extracted values for later use in the returned status object.

Design for class “PayOrderService”

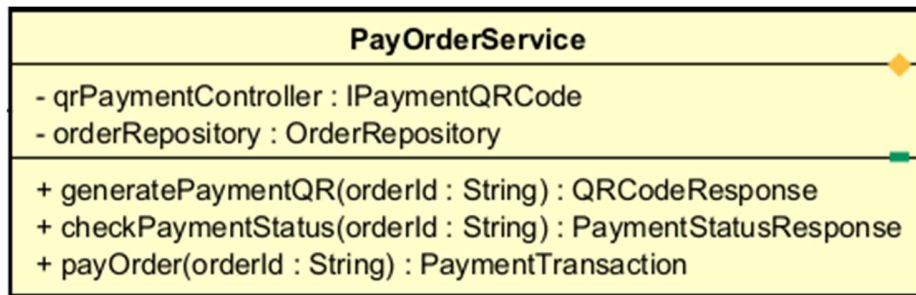


Table 1. Attribute design

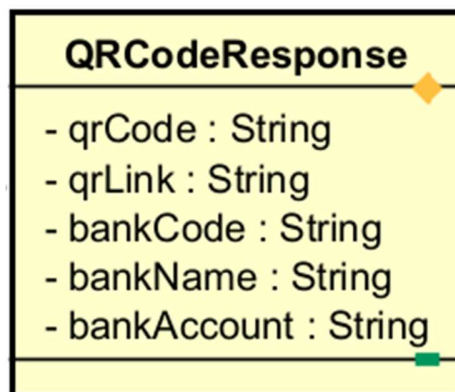
#	Name	Data type	Default value	Description
1	qrPaymentController	IPaymentQRCode		Interface used to communicate with QR payment subsystem (e.g. VietQR).
2	orderRepository	OrderRepository		Repository used to query and update order information in database.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	generatePaymentQR	QRCodeResponse	Generate QR code information for the given order.
2	checkPaymentStatus	PaymentStatusResponse	Check payment result of the given order.
3	payOrder	PaymentTransaction	Complete payment process and save payment transaction.

Parameter:

- generatePaymentQR(orderId : String) : QRCodeResponse
 - orderId: the ID of the order to generate payment QR
- checkPaymentStatus(orderId : String) : PaymentStatusResponse
 - orderId: the ID of the order to check payment status
- payOrder(orderId : String) : PaymentTransaction
 - orderId: the ID of the order to complete payment

Design for class “QRCodeResponse”**Table 1. Attribute design**

#	Name	Data type	Default value	Description
1	qrCode	String		QR code string returned from VietQR API, used for displaying or generating the QR image.
2	qrLink	String		Link to view the generated QR code.

3	bankCode	String		Code of the receiving bank.
4	bankName	String		Name of the receiving bank.
5	bankAccount	String		Bank account number used for payment.

Design for class “QRGenerateRequest”

QRGenerateRequest	
- bankCode : String - bankAccount : String - userBankName : String - content : String - qrType : int - amount : long - orderId : String - transType : String	
+ QRGenerateRequest(bankCode : String, bankAccount : String, userBankName : String, content : String, amount : long, orderId : String) + buildRequestString() : String	

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	bankCode	String		Code of the receiving bank used for VietQR payment generation.
2	bankAccount	String		Bank account number used to receive payment.
3	userBankName	String		Name of the bank account owner.
4	content	String		Payment transfer content / transaction description.

5	qrType	int		Type of QR code (dynamic, static, or semi-dynamic).
6	amount	long		Payment amount of the order.
7	orderId	String		ID of the order associated with the payment.
8	transType	String		Type of transaction (e.g. credit transaction).

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	QRGenerateRequest	constructor	Initialize request object with payment information for VietQR API.
2	buildRequestString	String	Convert request attributes into request string / JSON format for VietQR API call.

Parameter:

- QRGenerateRequest(bankCode : String, bankAccount : String, userBankName : String, content : String, amount : long, orderId : String)
 - bankCode: code of the receiving bank
 - bankAccount: bank account number for payment
 - userBankName: account owner name
 - content: payment transfer content
 - amount: payment amount
 - orderId: ID of the corresponding order

Method:

buildRequestString() : String

- Converts all stored attribute values into the request format required by the VietQR API.
- Constructs a formatted JSON / request string containing payment and bank information.
- Returns the final request string to be sent by the boundary layer.

Design for class “QRAccessTokenResponse”

QRAccessTokenResponse
- accessToken : String - tokenType : String - expiresIn : int
+ parseResponseString(response : String) : void + getAccessToken() : String + getExpiresIn() : int + isValid() : boolean

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	accessToken	String		Access token returned from VietQR API, used for authenticated requests.
2	tokenType	String		Type of token returned by VietQR (e.g. Bearer).
3	expiresIn	int		Token expiration time returned by VietQR API, usually in seconds.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	parseResponseString	void	Parse raw VietQR token response string and extract token information.
2	getAccessToken	String	Return the parsed access token.
3	getExpiresIn	int	Return the token expiration time.
4	isValid	boolean	Check whether the parsed token response is valid.

Parameter:

- parseResponseString(response : String) : void
 - response: response string returned from VietQR token API

Design for class “QRAccessTokenRequest”

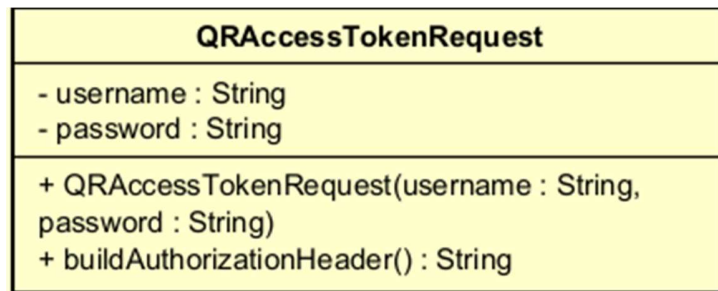


Table 1. Attribute design

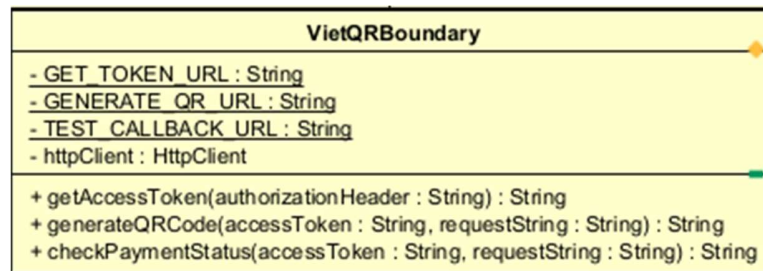
#	Name	Data type	Default value	Description
1	username	String		VietQR account username used to request access token.
2	password	String		VietQR account password used to request access token.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	QRAccessTokenRequest	constructor	Initialize token request object with username and password.
2	buildAuthorizationHeader	String	Build authorization header string for VietQR token API request.

Parameter:

- QRAccessTokenRequest(username : String, password : String)
 - username: VietQR account username
 - password: VietQR account password

Design for class “VietQRBoundary”**Table 1. Attribute design**

#	Name	Data type	Default value	Description
1	GET_TOKEN_URL	String		URL of the VietQR API endpoint used to request an access token.
2	GENERATE_QR_URL	String		URL of the VietQR API endpoint used to generate a payment QR code.
3	TEST_CALLBACK_URL	String		URL of the VietQR API endpoint used to simulate or check payment callback status.

4	httpClient	HttpClient		HTTP client used to send requests to VietQR APIs.
---	------------	------------	--	---

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	getAccessToken	String	Send a request to VietQR to obtain an access token and return the raw response string.
2	generateQRCode	String	Send a QR generation request to VietQR and return the raw response string.
3	checkPaymentStatus	String	Send a payment status or callback request to VietQR and return the raw response string.

Parameter:

getAccessToken(authorizationHeader : String) : String

- authorizationHeader: authorization header built from VietQR username and password, used to request an access token.

generateQRCode(accessToken : String, requestString : String) : String

- accessToken: valid VietQR access token used for authenticated API calls.
- requestString: formatted request payload containing bank and payment information for QR generation.

checkPaymentStatus(accessToken : String, requestString : String) : String

- accessToken: valid VietQR access token used for authenticated API calls.
- requestString: formatted request payload used to query or simulate payment status.

Method:

getAccessToken(authorizationHeader : String)

- Sends an HTTP request to the VietQR token API using the provided authorization header.
- Receives the raw response containing access token information.
- Returns the response string for further parsing.

generateQRCode(accessToken : String, requestString : String)

- Sends an HTTP request to the VietQR QR generation API using the access token and request payload.
- Receives the raw response containing generated QR information.
- Returns the response string for further parsing.

checkPaymentStatus(accessToken : String, requestString : String)

- Sends an HTTP request to the VietQR payment status or callback API using the given token and payload.
- Receives the raw response containing transaction status information.
- Returns the response string for further parsing.

Design for class “IPaymentQRCode”



Table 2. Operation design

#	Name	Return type	Description (purpose)
1	generateQRCode	QRCodeResponse	Generate a QR code for the given order and return QR payment information.
2	checkPaymentStatus	PaymentStatusResponse	Check the payment status of the given order and return the payment result.

Parameter

generateQRCode(order : Order) : QRCodeResponse

- order: order object used to generate the payment QR code based on order information such as order ID and total amount.

checkPaymentStatus(order : Order) : PaymentStatusResponse

- order: order object used to check the payment status of the corresponding transaction.

Method:

generateQRCode(order : Order)

- Defines the method for generating a payment QR code based on the provided order.
- Must be implemented by the concrete payment QR controller class.
- Returns the QR payment information for display to the customer.

checkPaymentStatus(order : Order)

- Defines the method for checking the payment status of the provided order.
- Must be implemented by the concrete payment QR controller class.
- Returns the current payment status information.

Design for class “QRTestCallbackRequest”

QRTestCallbackRequest	
-	bankAccount : String content : String amount : long bankCode : String transType : String
+	QRTestCallbackRequest(bankAccount : String, content : String, amount : long, bankCode : String, transType : String) buildRequestString() : String

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	bankAccount	String		Bank account number used in the callback simulation request.
2	content	String		Payment content or transaction description associated with the order.
3	amount	long		Payment amount used in the callback request.
4	bankCode	String		Bank code of the receiving account.
5	transType	String		Type of transaction used in the callback simulation (e.g., incoming transfer).

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	QRTestCallbackRequest	Constructor	Initialize the callback request object with payment and bank information.
2	buildRequestString	String	Build the request payload string for the VietQR test callback API.

Parameter:

QRTestCallbackRequest(bankAccount : String, content : String, amount : long, bankCode : String, transType : String)

- bankAccount: bank account number used in the callback request.
- content: payment description or order content.
- amount: transaction amount used for callback simulation.
- bankCode: bank code of the receiving account.
- transType: transaction type for the callback request.

Method:

QRTestCallbackRequest(bankAccount : String, content : String, amount : long, bankCode : String, transType : String)

- Initializes the callback request object with all required transaction information.
- Stores the provided values into the corresponding attributes.
- Prepares the object to be converted into a request payload string.

buildRequestString() : String

- Converts all stored attribute values into the format required by the VietQR callback API.
- Builds a formatted JSON / request string containing callback simulation data.
- Returns the final request string to be sent by the boundary layer.

Design for class “PaymentQRController”

PaymentQRController	
- payOrderService : PayOrderService	
+ initiateQRPayment(orderId : String) : ResponseEntity<QRCodeResponse>	
+ checkPaymentStatus(orderId : String) : ResponseEntity<PaymentStatusResponse>	
+ confirmPayment(orderId : String) : ResponseEntity<PaymentTransactionResponse>	

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	payOrderService	PayOrderService		Service object used to handle the Pay Order QR payment business logic.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	initiateQRPayment	ResponseEntity	Initiate QR payment for the specified order and return QR payment information to the client.
2	checkPaymentStatus	ResponseEntity	Check the payment status of the specified order and return the result to the client.
3	confirmPayment	ResponseEntity	Confirm payment completion for the specified order and return transaction details.

Parameter:

initiateQRPayment(orderId : String) : ResponseEntity<QRCodeResponse>

- orderId: the ID of the order used to initiate QR payment and generate the payment QR code.

checkPaymentStatus(orderId : String) : ResponseEntity<PaymentStatusResponse>

- orderId: the ID of the order used to check its payment status.

confirmPayment(orderId : String) :
ResponseEntity<PaymentTransactionResponse>

- orderId: the ID of the order used to confirm successful payment and retrieve transaction details.

Method:

initiateQRPayment(orderId : String)

- Calls `payOrderService.generatePaymentQR(orderId)` to generate QR payment information.
- Wraps the returned result into a `ResponseEntity`.
- Returns the response to the client.

checkPaymentStatus(orderId : String)

- Calls `payOrderService.checkPaymentStatus(orderId)` to retrieve the payment status.
- Wraps the returned result into a `ResponseEntity`.
- Returns the response to the client.

confirmPayment(orderId : String)

- Calls `payOrderService.payOrder(orderId)` to finalize payment.
- Converts transaction information into response format.
- Returns the transaction response to the client.

Design for class “PaymentTransactionResponse”

PaymentTransactionResponse
- transactionId : String - paymentMethod : String - amount : double - status : String - transactionTime : int

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	transactionId	String		Unique identifier of the payment transaction.
2	paymentMethod	String		Payment method used for the transaction (e.g., QR, credit card).
3	amount	double		Total amount paid in the transaction.
4	status	String		Current status of the transaction (e.g., success, failed, pending).
5	transactionTime	int		Time when the transaction was completed or recorded.